

# Parrainage et commissions — guide d'intégration mobile

Ce que le parrain voit, et ce que le filleul saisit. Toutes les réponses ci-dessous sont issues d'**appels réels** sur l'environnement de développement.  
Référence : issue #246. Complète le [guide des abonnements](#).

## 1. La règle, en une phrase

Si un code valide — qui n'est pas celui de l'acheteur — est saisi au moment de l'achat, son propriétaire reçoit un pourcentage du montant payé. Sinon, rien n'est versé à personne.

Le code de parrainage servait jusqu'ici au **suivi** : savoir qui amène des utilisateurs. Il devient rémunérateur, mais uniquement par ce geste précis.

| au moment de l'achat                          | résultat                            |
|-----------------------------------------------|-------------------------------------|
| code valide, différent de celui de l'acheteur | le propriétaire du code est crédité |
| code inconnu                                  | rien                                |
| code de l'acheteur lui-même                   | rien                                |
| champ laissé vide                             | rien                                |

Il n'y a **aucun autre chemin**. Avoir un parrain d'inscription — quelqu'un dont le code avait servi à la création du compte — n'ouvre **aucun droit** si ce code n'est pas ressaisi à l'achat.

### Ce que cela implique pour l'application

Le **champ code est le déclencheur, pas un détail de formulaire**. Enfoui, mal libellé ou laissé vide par défaut, il ne se passe rien : ni erreur, ni avertissement, ni commission. Prévoyez-le visible sur l'écran d'achat, avec un libellé qui dit ce qu'il fait — « Un code de parrainage ? Son propriétaire sera récompensé » plutôt qu'un champ nu.

Deux surfaces côté application :

- le **champ code** à l'achat, proposé à **tous** les acheteurs ;
- un **écran « Mes parrainages »** : filleuls et commissions perçues.

Le versement lui-même est automatique : il part du serveur à l'encaissement, sans appel supplémentaire de votre part.

Le taux et l'activation se règlent depuis le back-office (**Abonnements** → **Commission parrainage**) sans déploiement.

### ⚠ La commission est désactivée au lancement

Elle est livrée à **0 %**, **désactivée**. Un taux non arbitré qui verserait de l'argent réel dès le premier abonnement serait difficile à rattraper. L'administration l'active depuis le back-office.

Conséquence : `commissions.total` vaudra **0** pour tout le monde au début. Ce n'est pas une panne — prévoyez un état vide qui a du sens plutôt qu'un « 0 XOF » sec.

**Authentification.** Jeton JWT requis. **Scope pays** habituel.

## 2. Le code du parrain — GET /utilisateurs/code-parrainage

Inchangé, déjà en place :

```
{ "code_parrainage": "KLVC1" }
```

C'est ce code que le parrain partage. Six caractères, majuscules.

3. Mes parrainages — GET /abonnements/mes-parrainages?country=benin

```
{
  "filleuls": [
    { "uuid": "d7bfb703-...", "nom": "T246", "prenom": "f2", "inscrit_le": "2026-09-06T08:32:25.802Z" },
    { "uuid": "6b3f9506-...", "nom": "T246", "prenom": "f1", "inscrit_le": "2026-09-06T08:32:24.601Z" }
  ],
  "nombre_filleuls": 2,
  "filleuls_abonnes": 2,
  "commissions": { "nombre": 2, "total": 400 }
}
```

| champ              | sens                                                 |
|--------------------|------------------------------------------------------|
| filleuls           | 100 derniers inscrits, du plus récent au plus ancien |
| nombre_filleuls    | taille de la liste renvoyée                          |
| filleuls_abonnes   | filleuls dont l'abonnement a produit une commission  |
| commissions.nombre | versements effectués                                 |
| commissions.total  | montant cumulé, dans la devise du wallet             |

**L'email des filleuls n'est pas exposé** — un parrain n'a pas à récupérer les adresses des gens qu'il a amenés. Affichez prénom et nom.

Le montant atterrit dans le wallet du parrain : les écrans wallet existants (solde, transactions, retrait) le montrent sans modification. Les transactions de commission portent `reward_source_type_code = "PARRAINAGE_ABONNEMENT"` .

4. Saisir un code à la souscription

POST /abonnements/souscrire accepte un `code_parrainage` **optionnel** :

```
{ "plan_uuid": "574612e9-...", "code_parrainage": "KLVCM1", "pays": "benin" }
```

Le champ est **facultatif** : l'omettre souscrit normalement, sans commission.

Ce que la réponse dit — et ne dit pas

`parrain_id` porte l'identifiant du bénéficiaire retenu, ou `null` :

```
{ "uuid": "abo-...", "statut": "EN_ATTENTE", "parrain_id": 26754, "montant_paye": 0 }
```

| réponse                         | interprétation                                                        |
|---------------------------------|-----------------------------------------------------------------------|
| <code>parrain_id</code> non nul | le code a été accepté, son propriétaire sera crédité à l'encaissement |
| <code>parrain_id: null</code>   | aucun bénéficiaire — code absent, inconnu, ou celui de l'acheteur     |

**C'est le seul retour dont vous disposez.** Un code invalide ne déclenche **aucune erreur** : la souscription réussit, `parrain_id` vaut simplement `null` . C'est délibéré — bloquer un paiement pour une faute de frappe sur un champ facultatif serait absurde.

 **Ne confirmez donc jamais « code pris en compte » sur la seule foi du 201** . Vérifiez `parrain_id` . Si vous voulez signaler une saisie erronée, c'est le moment : `parrain_id: null` alors que l'utilisateur a saisi quelque chose signifie que le code n'a rien donné.

Deux pièges

**Ne pré-remplissez pas avec le code du parrain d'inscription.** La tentation est naturelle, mais cela restaure côté client une règle que le serveur a justement retirée, et paie quelqu'un qui n'a rien fait pour cette vente. Si le produit le veut malgré tout, que ce soit une décision explicite — le serveur ne le fera jamais de lui-même.

**Ne présentez pas l'opération comme un « changement de parrain ».** `utilisateurs.parrain_id` n'est ni lu ni modifié par ce chemin ; il reste la donnée d'acquisition, exploitée par les statistiques. Le code saisi ne vaut que pour l'abonnement en cours.

## 5. Quand la commission part

---

À l'**encaissement** de l'abonnement, pas à la souscription. Aujourd'hui c'est un administrateur qui confirme le paiement ; avec l'intégration du paiement en ligne, ce sera le webhook du prestataire.

```
souscription → abonnement EN_ATTENTE, parrain figé
               ↓ paiement encaissé
activation   → abonnement ACTIF + commission créditée au parrain
```

Le parrain est **figé à la souscription**. S'il change entre-temps, la commission va bien à celui qui était rattaché au moment de l'achat.

Un versement peut échouer sans conséquence pour le filleul : son abonnement s'active quoi qu'il arrive. Les commissions en souffrance sont rattrapées par l'administration.

**Aucune notification spécifique** n'est émise pour l'instant côté commission — le parrain la découvre sur son wallet ou sur l'écran « Mes parrainages ».

---

## 6. Ce que l'écran doit faire

---

**Un état vide qui explique.** Sans filleul : « Partagez votre code, gagnez sur chaque abonnement ». Avec des filleuls mais 0 commission : « Vos filleuls n'ont pas encore souscrit » — et non « 0 XOF », qui se lit comme une panne.

**Rendre le code partageable.** Bouton copier + partage système. C'est le seul geste qui fait vivre la fonctionnalité.

**Ne pas promettre de montant.** Le taux est réglable côté serveur et vaut 0 au lancement : n'affichez pas « gagnez 10 % » en dur.

**Renvoyer vers le wallet.** La commission y est visible avec le reste ; un lien évite de dupliquer un historique.

---

## 7. Exemple Flutter

```
class ParrainageApi {
  ParrainageApi(this._client, {required this.baseUrl, required this.token, required this.pays});
  final http.Client _client;
  final String baseUrl, token, pays;

  Map<String, String> get _entetes =>
    {'Authorization': 'Bearer $token', 'Content-Type': 'application/json'};

  Future<Parrainages> mesParrainages() async {
    final r = await _client.get(
      Uri.parse('$baseUrl/abonnements/mes-parrainages?country=$pays'),
      headers: _entetes,
    );
    return Parrainages.fromJson(jsonDecode(r.body));
  }

  Future<String> monCode() async {
    final r = await _client.get(
      Uri.parse('$baseUrl/utilisateurs/code-parrainage?country=$pays'),
      headers: _entetes,
    );
    return (jsonDecode(r.body) as Map<String, dynamic>)['code_parrainage'] as String;
  }
}

class ResultatSouscription {
  const ResultatSouscription({required this.abonnement, required this.codeIgnore});
  final Abonnement abonnement;
  /// Un code a été saisi mais n'a désigné personne : inconnu, ou celui de
  /// l'acheteur. Aucune commission ne sera versée pour cet abonnement.
  final bool codeIgnore;
}

class Parrainages {
  const Parrainages({
    required this.filleuls,
    required this.filleulsAbonnes,
    required this.nombreCommissions,
    required this.totalCommissions,
  });

  final List<Filleul> filleuls;
  final int filleulsAbonnes, nombreCommissions;
  final num totalCommissions;

  /// Distinguer « pas encore de filleul » de « filleuls sans abonnement » :
  /// les deux valent 0 XOF mais n'appellent pas le même message.
  bool get aucunFilleul => filleuls.isEmpty;
  bool get aucuneCommission => filleuls.isNotEmpty && nombreCommissions == 0;
}
```

## Souscrire avec un code

```
Future<ResultatSouscription> souscrire(String planUuid, {String? codeParrainage}) async {
  final saisi = codeParrainage?.trim() ?? '';

  final r = await _client.post(
    Uri.parse('$baseUrl/abonnements/souscrire'),
    headers: _entetes,
    body: jsonEncode({
      'plan_uuid': planUuid,
      'pays': pays,
      // SEUL déclencheur de commission. Sans ce champ, personne n'est payé –
      // pas même le parrain d'inscription.
      if (saisi.isNotEmpty) 'code_parrainage': saisi.toUpperCase(),
    }),
  );
  if (r.statusCode ~/ 100 != 2) throw ApiException(messageErreur(jsonDecode(r.body)));

  final abonnement = Abonnement.fromJson(jsonDecode(r.body));

  // Un code invalide ne lève AUCUNE erreur : le 201 ne prouve rien sur le
  // code. `parrain_id` est le seul retour exploitable.
  final codeRetenu = abonnement.parrainId != null;
  return ResultatSouscription(
    abonnement: abonnement,
    // Distinguer « rien saisi » de « saisi mais sans effet » : seul le second
    // mérite un message à l'utilisateur.
    codeIgnore: saisi.isNotEmpty && !codeRetenu,
  );
}
```

À l'écran :

```
final res = await api.souscrire(plan.uuid, codeParrainage: champCode.text);

if (res.codeIgnore) {
  // Ne bloque pas le paiement : la souscription est déjà passée.
  afficherInfo('Ce code de parrainage n'a pas été reconnu. Votre abonnement se poursuit.');
```

## 8. Rappels

**La commission est à 0 % et désactivée au lancement.** N'affichez aucun taux en dur, et prévoyez l'état « aucune commission ».

**Sans code valide saisi à l'achat, aucune commission n'est versée** — le parrain d'inscription ne suffit pas. Rendez le champ visible : c'est la saisie qui déclenche tout.

`parrain_id` dans la réponse de `souscrire` est le seul retour sur le code. `null` = aucun bénéficiaire ; un `201` ne prouve rien.

**Un code invalide n'échoue pas et ne paie personne.** Aucune erreur ne remonte — vérifiez `parrain_id` avant d'annoncer quoi que ce soit.

**Le code de l'acheteur lui-même est refusé** (auto-parrainage), silencieusement comme un code inconnu.

**Le versement suit le paiement, pas la souscription.** Un abonnement `EN_ATTENTE` porte déjà son `parrain_id`, mais n'a encore rien crédité.

**L'email des filleuls n'est pas exposé.** Prénom et nom seulement.

**La commission arrive dans le wallet existant** — pas de nouvel écran de solde à construire.